AWS COMMUNITY DAY MUMBAI 2026

Don’t Call the Model Twice

Verify globally, repair locally

Susmit Vengurlekar · Data Systems Architect · SkillRev

60 QUESTIONS · ONE COMPLIANCE MANUAL

Six batches. All green. The quiz is wrong.

Six model calls in parallel · every batch returned valid JSON · every batch passed its grounding check

	B1	B2	B3	B4	B5	B6
Topic A	✓	✓				
Topic B		✓!	✓		✓!	
Topic C				✓		✓
Policy 3			✓			
Policy 4	✕	✕	✕	✕	✕	✕

✕ Policy 4 — zero questions anywhere ✓! Batches 2 & 5 — the same question, twice

Nothing errored. Every batch passed. **So what do you retry?**

TWENTY SECONDS ABOUT ME

Susmit Vengurlekar

Data Systems Architect at SkillRev · 8+ years shipping software

I work on **ForQuiz** — training manuals, decks, and SOPs become knowledge checks a human reviews before anyone sees them.

forquiz.fusebytes.in · almost everywhere as **susmitpy**

WHAT "CORRECT" ACTUALLY MEANS

Four properties. Two of them no single call can see.

CHECKABLE PER CALL

Valid structure

Schema, option count, exactly one correct answer.

CHECKABLE PER CALL

Grounded in the source

Every question traceable to the uploaded manual.

ONLY AFTER THE JOIN

Blueprint coverage

Every required topic and policy has questions.

ONLY AFTER THE JOIN

No duplicates

Not within a batch — **across** all six.

A batch can satisfy everything it is able to check **and still be part of a wrong quiz.**

THE SHAPE OF IT

One handler, four steps, one back-edge

reference sites · PDFs & decks · "what to cover" instruction → ingest is out of scope today

AWS LAMBDA DURABLE FUNCTION

Plan
1 cheap call · batch specs

→

Generate ×6
Nova 2 Lite + Guardrails · parallel

→

Verify
deterministic · no model

→

Publish
after human approval

↩ Verify → repair only the failing batch IDs → Verify again

Your table · DynamoDB — questions, generation recipe, attempt number

Runtime-managed — checkpoint history — you do not build or operate this

BEFORE YOU BLAME THE VALIDATION

Green is a stricter check than you'd guess.

```
BATCH 2 — validation result
 schema pass 10 questions · 4 options · exactly 1 correct
 grounding 0.91 threshold 0.75 · Bedrock Guardrails
 in-batch dupes pass no repeats inside this batch
 → GREEN
```

Contextual grounding runs **on the model call itself** — not as a later pass.

Six batches cleared that bar. **Policy 4 is still empty.**

THE THESIS

Local validity is not global validity.

Coverage and cross-batch duplication are properties of the **set**. Nothing inside a single call can evaluate them — no matter how strict that call's checks are.

THE OBVIOUS MOVES

Three answers, all bad.

Regenerate all six

6 → 12 calls to fix one gap.

Five good batches come back *different*. Content a human already approved is silently replaced.

One giant call

Context isn't the problem — **Nova 2 Lite holds a million tokens.**

But 60 questions strain a 64K output cap, one bad question fails the whole call, and you've lost per-topic control entirely.

Hand-roll a resume handler

State table, resume entry point, poller, retry bookkeeping.

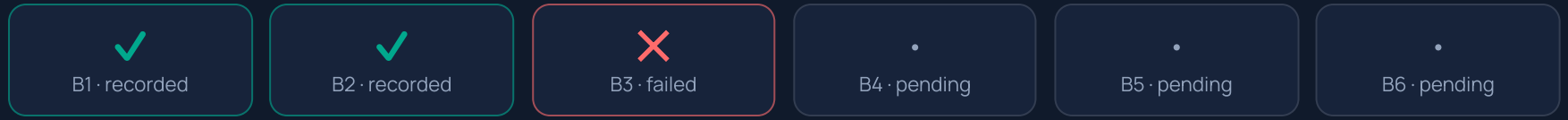
Every new failure mode grows it. You now maintain a workflow engine.

What you actually want: regenerate **only the failing batches**, hand them the verifier's findings, and loop until coverage holds.

THE MECHANISM

Replay isn’t resume.

The handler starts again **from line one**. Operations that already completed return their recorded result instead of running a second time.



STEP

Run business logic. Result is checkpointed.

REPLAY

Rerun from the top; completed steps hand back what they returned.

WAIT

Suspend for an event — up to a year — without paying for idle compute.

You get a **runtime-managed history** instead of the resume handler from three slides ago.

THE LOOP

Verify globally. Repair locally. Repeat.

Plan
batch specs + avoid-list

→

Generate ×6
parallel · each one a step

→

Verify
coverage · dupes · policy

→

Review
human · suspended, free

→

Publish
DynamoDB

↩ Verify fails → regenerate **only the failing batch IDs**, with the findings in the prompt → verify the whole set again.

Length is unknown at runtime

How many repair rounds you need depends on what verify finds. That's a data-dependent loop.

Finished batches never enter it

A batch that passed is not regenerated, not re-priced, not re-reviewed.

THE CODE

A data-dependent loop is just a for-loop.

```python
@durable_step
def generate(ctx, spec, recipe, feedback=None, attempt=1) -> str: ...    # → a DynamoDB key

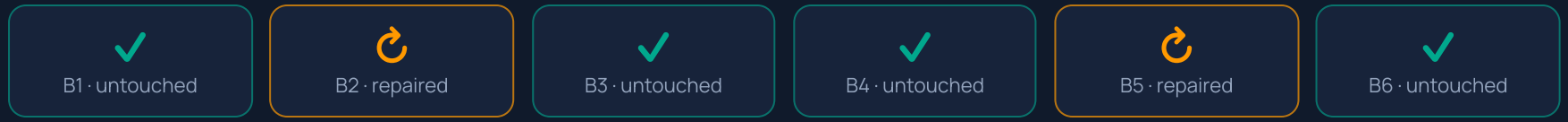
@durable_execution
def handler(event, context: DurableContext):
    recipe = event["recipe"]                                  # model + prompt + thresholds
    plan = context.step(plan_batches(event["doc"], recipe), name="plan")
    specs = {s.id: s for s in plan.specs}
    branches = [lambda ctx, s=s: ctx.step(generate(s, recipe), name=f"gen-{s.id}")
                for s in specs.values()]                      # parallel() wants ctx-takers
    refs = dict(zip(specs, context.parallel(branches).get_results()))

    for attempt in range(2, MAX_ATTEMPTS + 1):
        findings = context.step(verify(plan, refs), name=f"verify-a{attempt}")
        if findings.ok: break
        for bid in sorted(findings.failing):                  # stable order on replay
            refs[bid] = context.step(generate(specs[bid], recipe,
                findings.for_batch(bid), attempt), name=f"repair-{bid}-a{attempt}")

    gate = context.create_callback(name="review")             # suspends free · 72h timeout
```

WHAT IT COST

Nine calls, not thirteen.



Repair locally

1 plan + 6 generate + **2 repairs**

9 model calls

Regenerate everything

1 plan + 6 generate + **6 more**

13 model calls

With a cheap model, four saved calls is not the point. **Four batches a human already approved were never touched.**

BEFORE YOU BUILD THIS

Two things that will bite you.

1 · Replay runs your code again

Anything outside a step re-executes on every replay and must come out the same.

Into a step: `uuid4()`, `now()`, env reads, every network call.

Fine in orchestration: branching on step results, `sorted()` ordering.

Loop *length* may depend on step output. Iteration *order* may not.

2 · Your idempotency eats the repair

A conditional write keyed on `run + batch + index` is exactly right for replay safety.

Then a repair regenerates batch 2 — same run, same batch, same index. Same key.

The write is rejected as a duplicate. **The bad questions survive.**

```
sk = f"Q#{batch_id}#{q_idx}#A{attempt}"          # ← the attempt belongs in the key
table.put_item(Item={"pk": f"RUN#{run_id}", "sk": sk, "question": q},
               ConditionExpression="attribute_not_exists(sk)")
# replay of a finished write → same key → ConditionalCheckFailed → expected no-op ✓
# genuine repair, attempt 2   → new key  → writes; reader takes max(attempt)      ✓
```

THE OBVIOUS QUESTION

Why not Step Functions?

Durable functions

A loop whose shape depends on its own results, living inside application code.

Six batches, a verifier, and a feedback edge.

Step Functions

Massive fan-out, or a graph someone non-technical has to read.

Distributed Map, five services, an auditor.

The loop picked this one. **Scale or a non-engineer audience picks the other.** Running both is normal.

IF YOU REMEMBER THREE THINGS

Verify globally. Repair locally.

01

A batch cannot certify a property that spans batches. Coverage and duplication live after the join.

02

Checkpoint and replay turn the repair loop into a for-loop. Length can depend on results; order cannot.

03

Put the attempt in your idempotency key, or a correct write path will silently discard the fix.



Every batch is still the batch a human approved.

Nothing was bought twice.

Susmit Vengurlekar · almost everywhere as **susmitpy** · forquiz.fusebytes.in